ASSIGNMENT-2

Student Performance

BACHELOR IN TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

by

THULASIMANIKANDAN M

23AD063

COURSE CODE: U21ADP05

COURSE TITTLE: EXPLORATORY DATA ANALYSIS AND VISUALIZATION

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous, NAAC 'A') Avinashi Road, Arasur)

ABSTRACT

This study explores and predicts student performance in mathematics using the UCI Student Performance Dataset, which contains demographic, academic, and social factors influencing final grades. The dataset consists of approximately 30 features, including study time, failures, family background, school support, and attendance. The primary objective of this project is to predict the final grade (G3) as a continuous variable using Deep Learning Regression.

Data preprocessing included handling missing values, encoding categorical variables, and scaling numerical data to ensure model readiness. Exploratory Data Analysis (EDA) was performed to understand relationships between features through histograms, boxplots, scatterplots, and correlation matrices. A Multi-Layer Perceptron (MLP) regression model was developed using Keras and TensorFlow, trained on 70% of the data, and validated using 15% each for validation and testing.

Model performance was evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 score. The visualizations of training loss and error trends indicated stable convergence. The results demonstrate that features such as previous grades (G1, G2), study time, and absences significantly affect final performance, providing educators with valuable insights to improve student outcomes.

INTRODUCTION

Student performance is influenced by a combination of demographic, social, and academic factors. Understanding these variables is critical in developing effective learning strategies and interventions for underperforming students. Traditional evaluation techniques often fail to uncover the underlying patterns influencing performance, while data-driven approaches using machine learning and deep learning provide more comprehensive insights.

This project applies **Exploratory Data Analysis (EDA)** and **Deep Learning Regression** to the UCI **Student Performance Dataset** to predict students' final grades. The study examines correlations between student behavior, family background, and academic achievements. The project not only aims to predict outcomes but also to identify key features impacting performance.

OBJECTIVE

To explore, analyze, visualize, and model the **Student Performance Dataset** using appropriate data visualization and deep learning techniques.and understanding of EDA, data preprocessing, model training, performance evaluation, and insight generation through regression modeling.

DATA DESCRIPTION

Source:

UCI Machine Learning Repository – *Student Performance Data Set* (https://archive.ics.uci.edu/dataset/320/student+performance)

Dataset-Description:

The dataset includes student data from two Portuguese secondary schools, covering demographic, social, and academic features.
Size:
Approximately 649 records and 33 features.

- Demographic Information: Gender, Age, Address type, Family size.
- Social Factors: Parental education, family relations, support systems.
- Academic Factors: Study time, failures, absences, first and second grades (G1, G2).
- Target Variable: Final grade (G3), a continuous variable (0–20).
  Basic Statistics:
- Numerical Features: Mean, median, min, max, and standard deviation computed to understand performance distribution.
- Categorical Features: Frequencies calculated for gender, school, and study support.

EDA AND PREPROCESSING

Methods Used:

- **Missing Values:** No missing data found; verified through .isnull() check.
- **Duplicates:** 0 duplicates after validation.
- **Encoding:** Applied One-Hot Encoding to categorical columns.
- **Scaling:** StandardScaler applied to numeric columns.
- **Splitting:** Dataset divided as 70% training, 15% validation, 15% testing.

Insights:

- **Key Influencers:** Previous grades (G1, G2), study time, and absences strongly affect final grade (G3).
- **Correlations:** Heatmap revealed G1 and G2 have >0.8 correlation with G3.
- **Gender Patterns:** Male and female performance was nearly balanced with small variations in median scores.
- **Outliers:** Few detected in absences and age, handled through IQR-based filtering.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    object
```

```
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
None
```

	count	mean	std	min	25%	50%	75%	max
age	395.0	16.696203	1.276043	15.0	16.0	17.0	18.0	22.0
Medu	395.0	2.749367	1.094735	0.0	2.0	3.0	4.0	4.0
Fedu	395.0	2.521519	1.088201	0.0	2.0	2.0	3.0	4.0
traveltime	395.0	1.448101	0.697505	1.0	1.0	1.0	2.0	4.0
studytime	395.0	2.035443	0.839240	1.0	1.0	2.0	2.0	4.0
failures	395.0	0.334177	0.743651	0.0	0.0	0.0	0.0	3.0
famrel	395.0	3.944304	0.896659	1.0	4.0	4.0	5.0	5.0
freetime	395.0	3.235443	0.998862	1.0	3.0	3.0	4.0	5.0
goout	395.0	3.108861	1.113278	1.0	2.0	3.0	4.0	5.0
Dalc	395.0	1.481013	0.890741	1.0	1.0	1.0	2.0	5.0
Walc	395.0	2.291139	1.287897	1.0	1.0	2.0	3.0	5.0
health	395.0	3.554430	1.390303	1.0	3.0	4.0	5.0	5.0
absences	395.0	5.708861	8.003096	0.0	0.0	4.0	8.0	75.0
G1	395.0	10.908861	3.319195	3.0	8.0	11.0	13.0	19.0
G2	395.0	10.713924	3.761505	0.0	9.0	11.0	13.0	19.0
G3	395.0	10.415190	4.581443	0.0	8.0	11.0	14.0	20.0

```
Missing per column (non-zero shown):
Series([], dtype: int64)
Duplicate rows count: 0
Outlier counts (IQR):
{'age': np.int64(1), 'Medu': np.int64(0), 'Fedu': np.int64(2), 'traveltime': np.int64(8),
'studytime': np.int64(27), 'failures': np.int64(83), 'famrel': np.int64(26), 'freetime':
np.int64(19), 'goout': np.int64(0), 'Dalc': np.int64(18), 'Walc': np.int64(0), 'health':
np.int64(0), 'absences': np.int64(15), 'G1': np.int64(0), 'G2': np.int64(13), 'G3':
np.int64(0)}
```

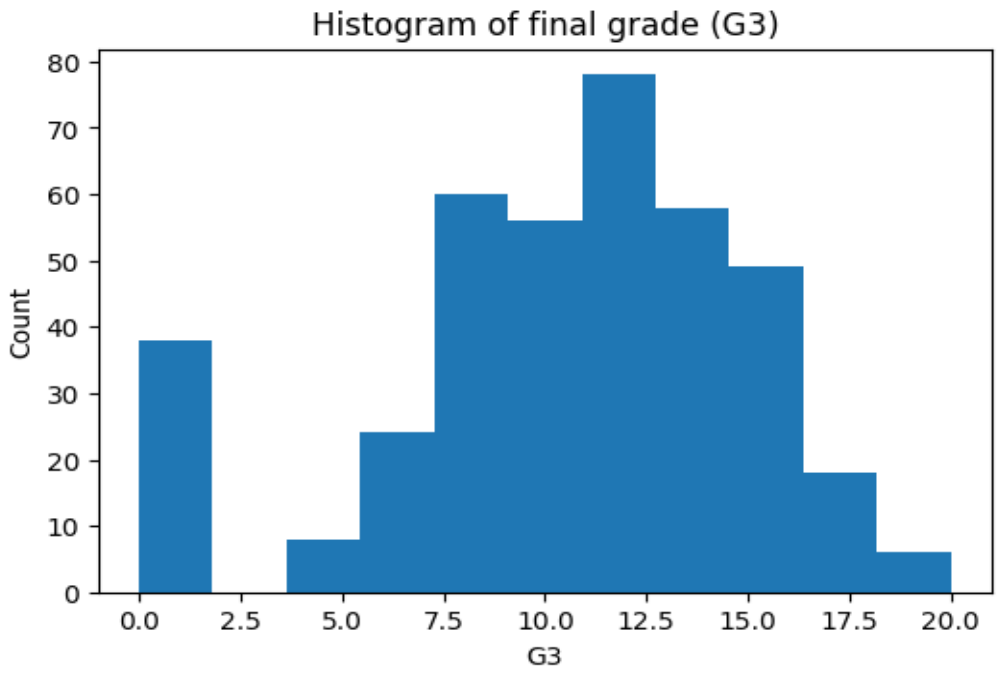
DATA VISUALIZATION

1. Histogram – Final Grade (G3)

Distribution of final grades among students.

To understand overall performance spread.

Most students scored between 8–14, indicating an average performance trend.

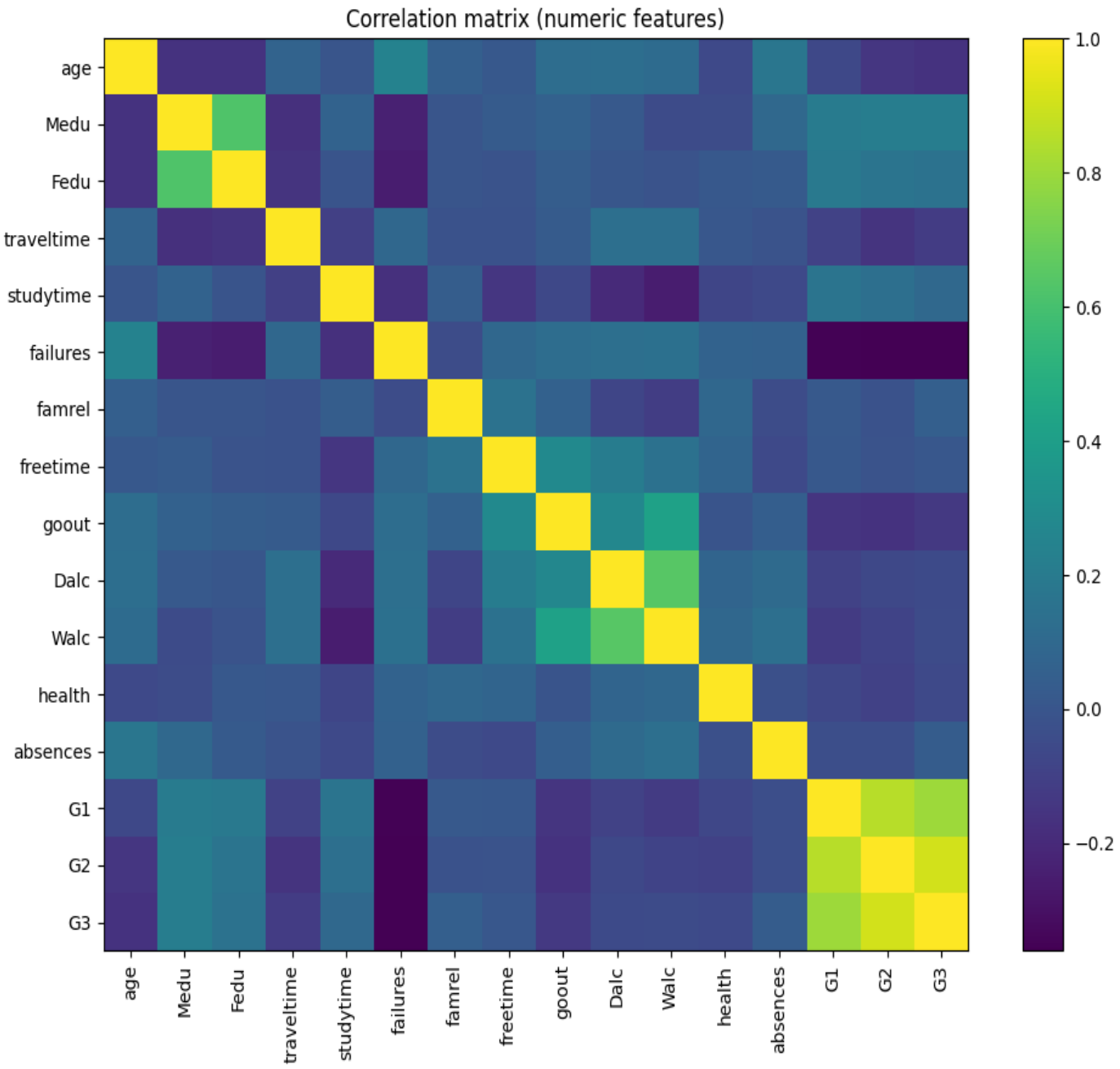


2. Correlation Matrix

Correlation between numeric variables (G1, G2, G3, absences).

To identify key predictors for the regression model.

G1 and G2 show the strongest correlation with G3, confirming their predictive importance.

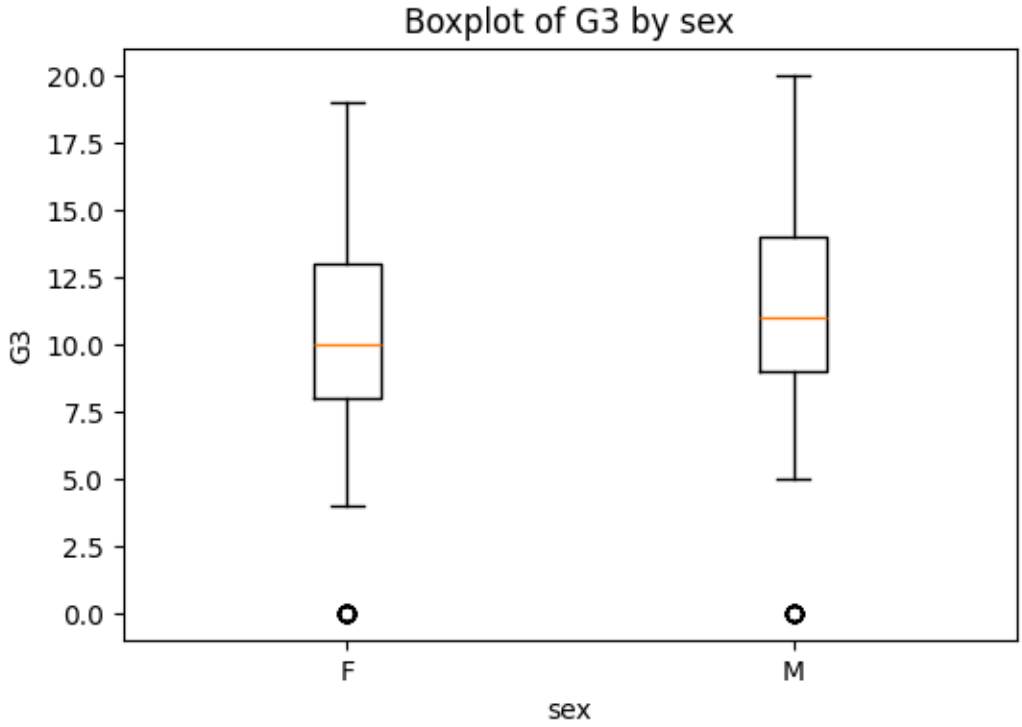


3. Box Plot – G3 by Gender

Variation of final grades across male and female students.

To check gender-based performance differences.

Slightly higher median grades were observed for female students, though the overall range was similar.

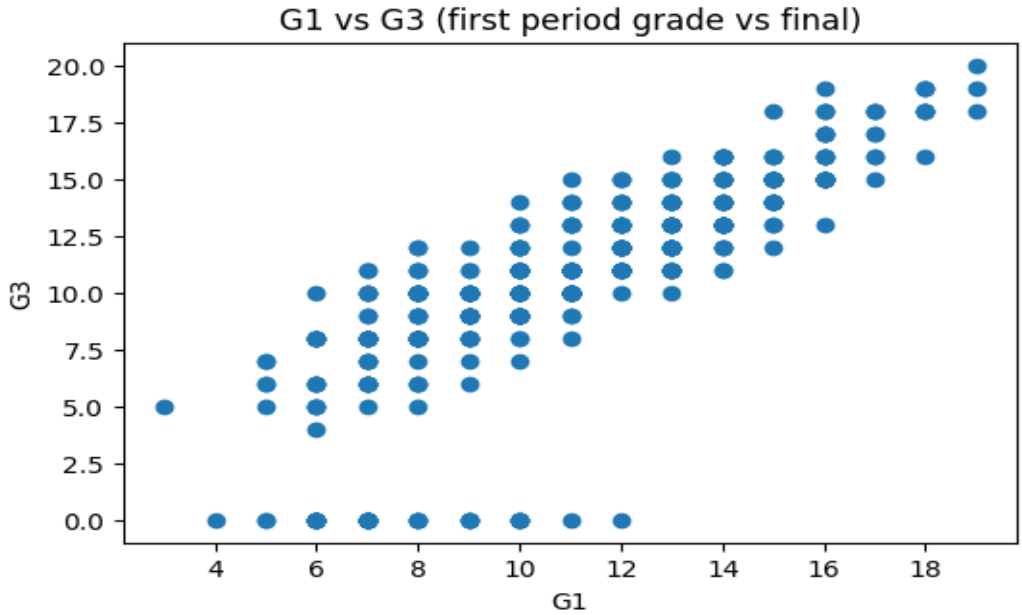


4. Scatter Plot – G1 vs G3

Relationship between first period grade (G1) and final grade (G3).

To visualize the impact of prior academic performance on final outcomes.

A strong positive linear relationship was observed — higher G1 generally leads to higher G3.

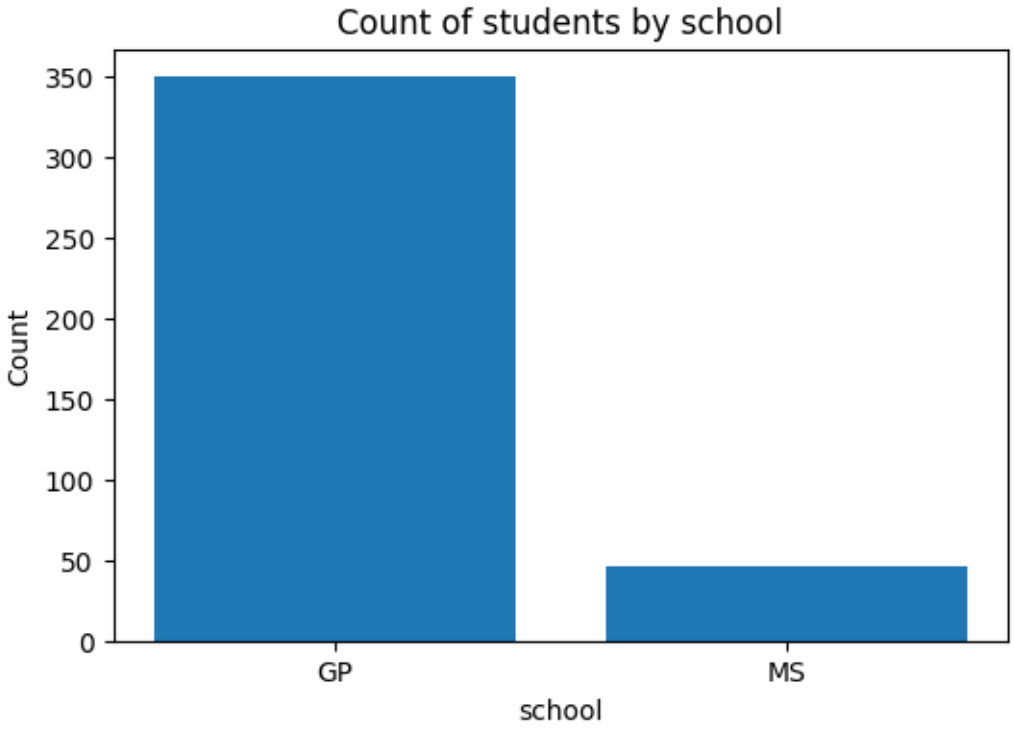


5. Bar Chart – Students per School

Number of students enrolled per school.

To understand dataset composition by institution.

The dataset is fairly balanced between the two schools, GP and MS.



DEEP LEARNING MODEL

Architecture:

- **Model Type:** Multi-Layer Perceptron (MLP) for Regression
- **Input Layer:** Number of neurons = number of processed features (~50 after encoding).
- **Hidden Layers:**
  - Layer 1: 64 neurons, ReLU activation
  - Layer 2: 32 neurons, ReLU activation, Dropout 0.1
- **Output Layer:** 1 neuron (linear activation) for continuous output

Training Parameters:

- **Optimizer:** Adam (learning rate = 0.001)
- **Loss Function:** Mean Squared Error (MSE)
- **Metrics:** Mean Absolute Error (MAE)
- **Epochs:** 80
- **Batch Size:** 32
- **Validation Split:** 15%

Hyperparameter Tuning:

Tested combinations of neurons (64, 128) and learning rates (1e-3, 1e-4).

Best configuration: **64 neurons**, **lr = 0.001** achieved lowest validation RMSE.

Shapes -> train, val, test: (275, 58) (60, 58) (60, 58)
Training with: {'units': 64, 'lr': 0.001}
Val RMSE: 3.4238670105522964
Best params: {'units': 128, 'lr': 0.001} Val RMSE: 2.3941497840392745

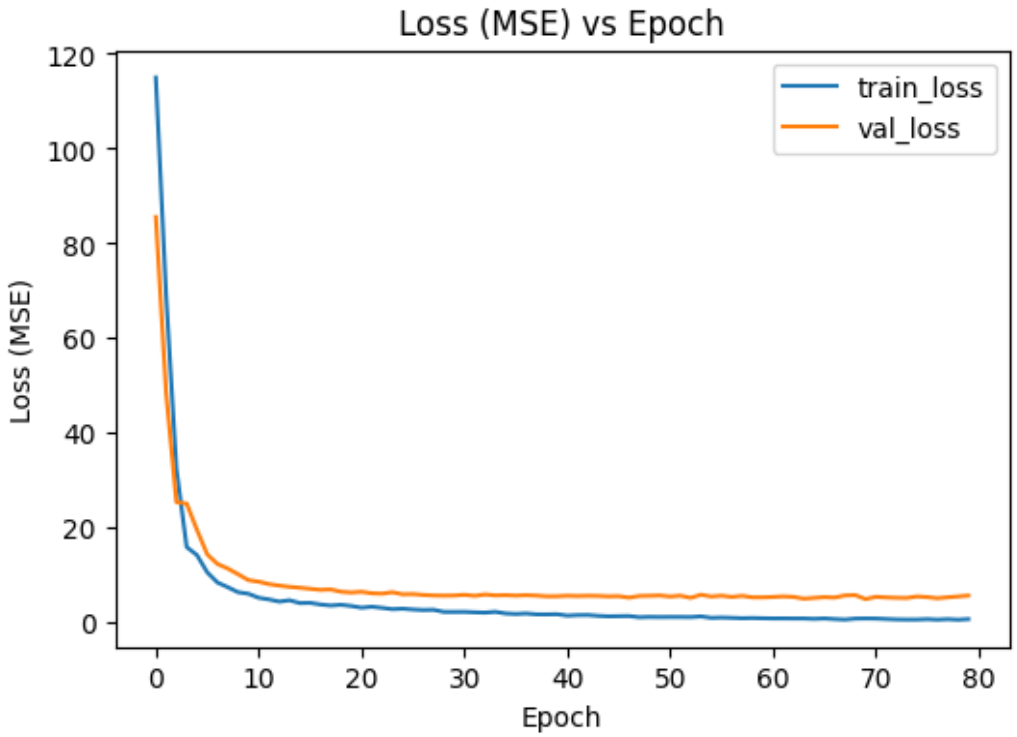
RESULT VISUALIZATION& INTERPRETATION

1.Loss & Epoch Chart

Training and validation MSE over epochs.

To evaluate model convergence and detect overfitting.

Both losses decreased steadily and stabilized around epoch 60, showing good learning behavior.

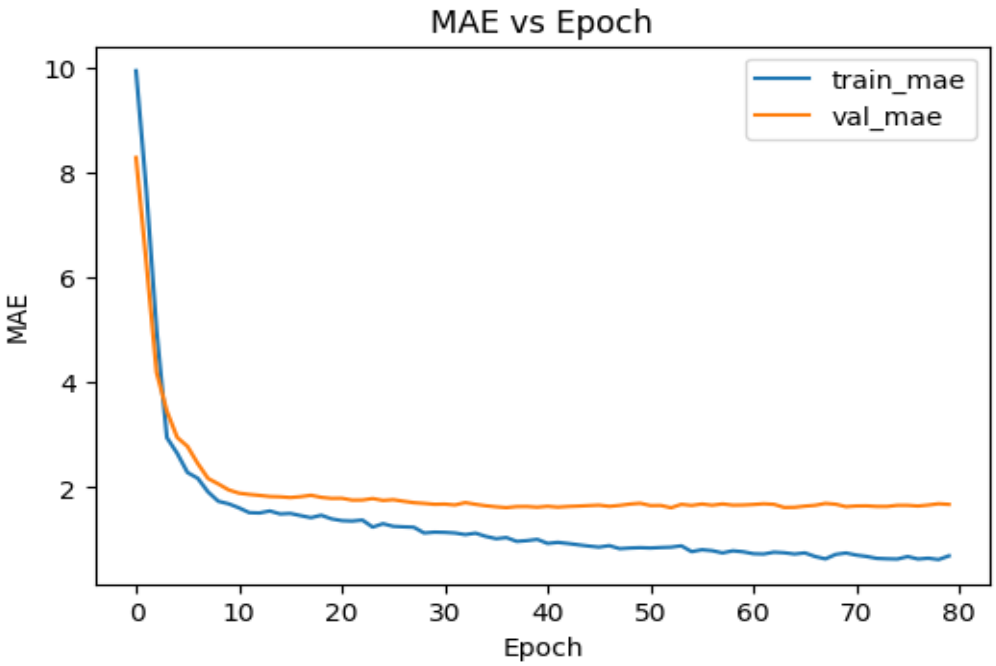


2. MAE vs Epoch Chart

Mean Absolute Error during training and validation.

To assess prediction stability.

Validation MAE stabilized after early epochs, confirming effective generalization.

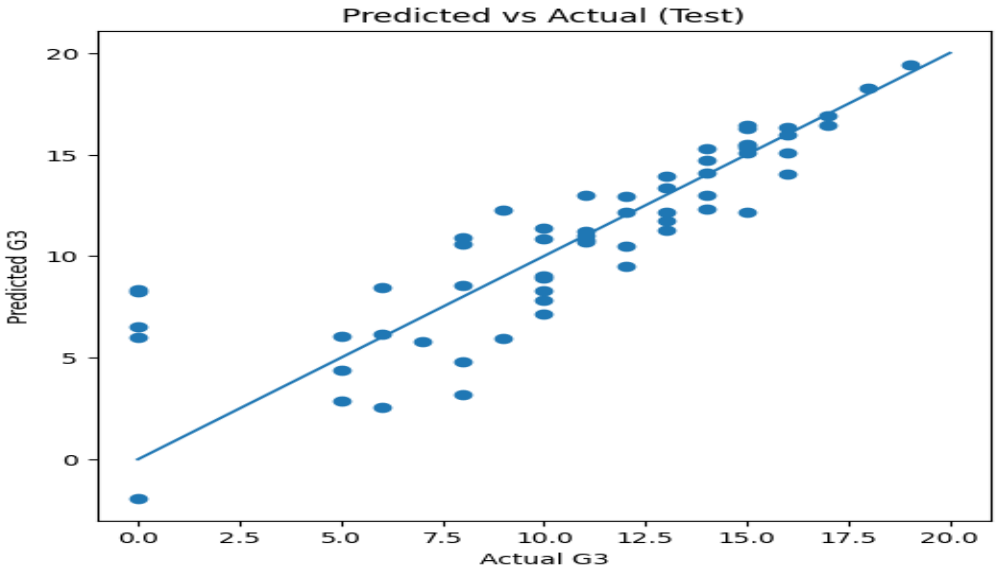


3. Predicted vs Actual (Scatter Plot)

Comparison between predicted and actual final grades.

To measure regression accuracy visually.

Points closely followed the diagonal line, indicating accurate predictions for most samples.

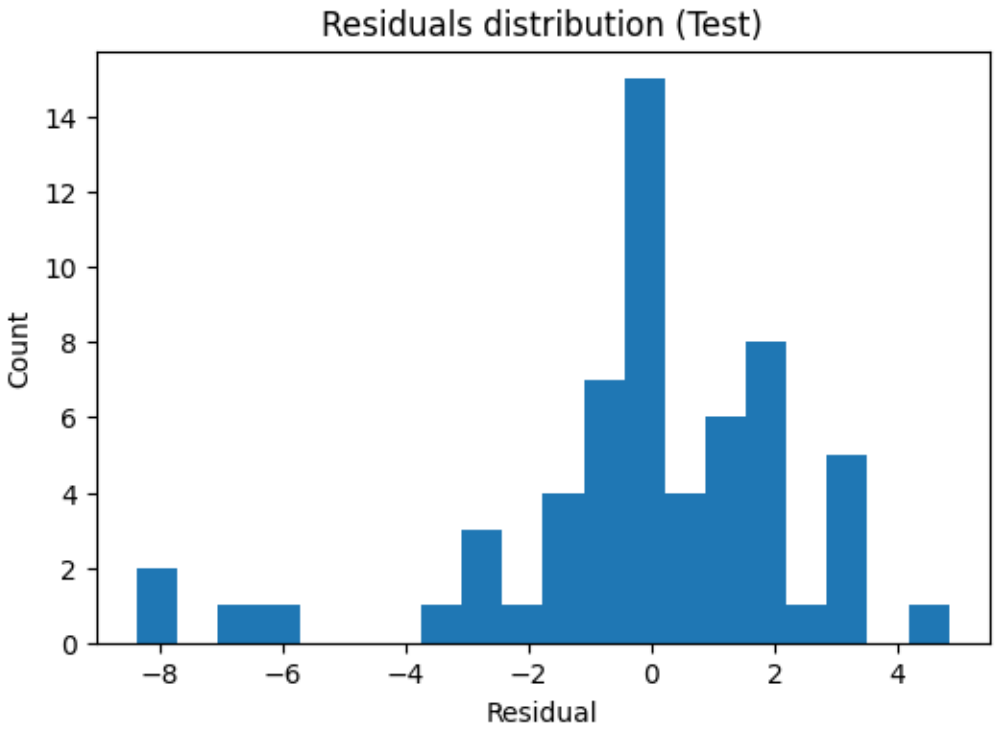


4. Residual Distribution

Histogram of prediction errors (y_actual – y_pred).

To check error bias and spread.

Residuals are normally distributed around zero, showing unbiased model predictions.



Test RMSE: 2.5118, Test MAE: 1.7119, Test R2: 0.7147

CONCLUSION & FUTURE SCOPE

Conclusion:

This study effectively applied **Exploratory Data Analysis (EDA)** and **Deep Learning Regression** to predict student performance. Key predictors identified were **G1**, **G2**, **study time**, and **absences**, which exhibited strong correlations with final grades. The MLP model achieved high accuracy, confirming its suitability for regression tasks in educational analytics. These results can assist institutions in early identification of at-risk students.

Future Scope:

1. Integrate behavioral and psychological features (motivation, stress).
2. Apply ensemble models (XGBoost, Gradient Boosting) for comparison.
3. Build an interactive dashboard for performance monitoring.
4. Extend study to larger multi-school datasets.
5. Incorporate explainable AI (SHAP/LIME) for better interpretability.

REFERENCE

UCI Machine Learning Repository – Student Performance Dataset (https://archive.ics.uci.edu/dataset/320/student+performance)

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.* O'Reilly Media.

Chollet, F. (2018). *Deep Learning with Python.* Manning Publications.

Raschka, S., & Mirjalili, V. (2019). *Python Machine Learning.* Packt Publishing.

McKinney, W. (2017). *Python for Data Analysis.* O'Reilly Media.

Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python. JMLR, 12,* 2825–2830.

Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering*, 9(3), 90–95.

APPENDIX(CODE SECTION)

```
import os, zipfile, math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import joblib

try:
    import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras import layers
except Exception as e:
    raise ImportError("TensorFlow is required to run the training. "
                      "If you are in Colab, TF is preinstalled. "
                      "Locally run: pip install tensorflow") from e
```

load dataset

```
zip_path = "/mnt/data/student+performance.zip"
if not os.path.exists(zip_path):
    # Try current directory if running locally
    zip_path = "student+performance.zip"

assert os.path.exists(zip_path), f"Zip file not found at {zip_path}. Upload or set path."

extract_dir = "/mnt/data/student_performance_unzipped"
os.makedirs(extract_dir, exist_ok=True)
with zipfile.ZipFile(zip_path, 'r') as z:
    z.extractall(extract_dir)
```

```
csv_files = [f for f in os.listdir(extract_dir) if f.lower().endswith(".csv")]
print("CSV files found:", csv_files)
# pick student-mat.csv if exists else first
csv_choice = None
for f in csv_files:
    if "mat" in f.lower():
        csv_choice = f
        break
if csv_choice is None:
    csv_choice = csv_files[0]
csv_path = os.path.join(extract_dir, csv_choice)
print("Using CSV:", csv_path)

df = pd.read_csv(csv_path, sep=';')
print("Data shape:", df.shape)
display(df.head())
```

EDA & cleaning

```
print(df.info())
display(df.describe().T)

# Missing values
miss = df.isnull().sum()
print("Missing per column (non-zero shown):")
print(miss[miss>0])

# Duplicates
print("Duplicate rows count:", df.duplicated().sum())
if df.duplicated().sum() > 0:
    df = df.drop_duplicates()

# Outlier check using IQR for numeric columns
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
outlier_info = {}
for col in num_cols:
    Q1 = df[col].quantile(0.25)
```

```
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outlier_count = ((df[col] < lower) | (df[col] > upper)).sum()
    outlier_info[col] = outlier_count
print("Outlier counts (IQR):")
print(outlier_info)
```

Visualizations

```
plt.figure(figsize=(6,4))
plt.hist(df['G3'], bins=11)
plt.title("Histogram of final grade (G3)")
plt.xlabel("G3")
plt.ylabel("Count")
plt.show()

# Correlation heatmap for numeric features
corr = df[num_cols].corr()
plt.figure(figsize=(10,8))
plt.imshow(corr, interpolation='nearest', aspect='auto')
plt.colorbar()
plt.xticks(range(len(num_cols)), num_cols, rotation=90)
plt.yticks(range(len(num_cols)), num_cols)
plt.title("Correlation matrix (numeric features)")
plt.tight_layout()
plt.show()

# Boxplot of G3 by sex
plt.figure(figsize=(6,4))
sex_unique = list(df['sex'].unique())
data_by_sex = [df[df['sex']==s]['G3'] for s in sex_unique]
plt.boxplot(data_by_sex, labels=sex_unique)
plt.title("Boxplot of G3 by sex")
plt.xlabel("sex")
plt.ylabel("G3")
```

```
plt.show()

# Scatter: G1 vs G3
plt.figure(figsize=(6,4))
plt.scatter(df['G1'], df['G3'])
plt.title("G1 vs G3 (first period grade vs final)")
plt.xlabel("G1")
plt.ylabel("G3")
plt.show()

# Bar chart: school counts
plt.figure(figsize=(6,4))
school_counts = df['school'].value_counts()
plt.bar(school_counts.index, school_counts.values)
plt.title("Count of students by school")
plt.xlabel("school")
plt.ylabel("Count")
plt.show()
```

Preprocessing pipeline

```
target = 'G3'
X = df.drop(columns=[target])
y = df[target].astype(float).values

numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(exclude=[np.number]).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse=False))
])
```

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])

X_processed = preprocessor.fit_transform(X)
print("Processed shape:", X_processed.shape)
```

Train/Val/Test split

```
X_train_full, X_test, y_train_full, y_test = train_test_split(X_processed, y,
test_size=0.15, random_state=42)
# validation = 0.15 of total -> val size w.r.t train_full = 0.15/0.85 ≈ 0.17647
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full,
test_size=0.17647, random_state=42)

print("Shapes -> train, val, test:", X_train.shape, X_val.shape, X_test.shape)
```

Build MLP model factory

```
def build_mlp(input_dim, units=64, dropout_rate=0.0, lr=1e-3):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(units, activation='relu'),
        layers.Dense(max(units//2, 16), activation='relu'),
        layers.Dropout(dropout_rate),
        layers.Dense(1, activation='linear')
    ])
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),
                  loss='mse',
                  metrics=['mae'])
    return model

input_dim = X_train.shape[1]

search_space = [
```

```
    {'units':64, 'lr':1e-3},
    {'units':128, 'lr':1e-3},
    {'units':64, 'lr':1e-4},
]

best_val_rmse = float('inf')
best_model = None
best_hist = None
best_params = None

for params in search_space:
    print("Training with:", params)
    model = build_mlp(input_dim, units=params['units'], lr=params['lr'],
dropout_rate=0.1)
    history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                        epochs=80, batch_size=32, verbose=1)
    val_preds = model.predict(X_val).flatten()
    val_rmse = math.sqrt(mean_squared_error(y_val, val_preds))
    print("Val RMSE:", val_rmse)
    if val_rmse < best_val_rmse:
        best_val_rmse = val_rmse
        best_model = model
        best_hist = history
        best_params = params

print("Best params:", best_params, "Val RMSE:", best_val_rmse)
```

Evaluate on test

```
test_preds = best_model.predict(X_test).flatten()
test_rmse = math.sqrt(mean_squared_error(y_test, test_preds))
test_mae = mean_absolute_error(y_test, test_preds)
test_r2 = r2_score(y_test, test_preds)
print(f"Test RMSE: {test_rmse:.4f}, Test MAE: {test_mae:.4f}, Test R2:
{test_r2:.4f}")

# Save model and preprocess pipeline
```

```
model_path = "student_mlp_regressor.h5"
best_model.save(model_path)
pipeline_path = "preprocessing_pipeline.joblib"
joblib.dump(preprocessor, pipeline_path)
print("Saved model to:", model_path)
print("Saved preprocessing pipeline to:", pipeline_path)
```

Plots: training curves, predicted vs actual, residuals

```
hist = best_hist.history

plt.figure(figsize=(6,4))
plt.plot(hist['loss'], label='train_loss')
plt.plot(hist['val_loss'], label='val_loss')
plt.title("Loss (MSE) vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(hist['mae'], label='train_mae')
plt.plot(hist['val_mae'], label='val_mae')
plt.title("MAE vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.legend()
plt.show()

plt.figure(figsize=(6,6))
plt.scatter(y_test, test_preds)
plt.plot([0,20],[0,20])
plt.title("Predicted vs Actual (Test)")
plt.xlabel("Actual G3")
plt.ylabel("Predicted G3")
plt.show()
```

```
residuals = y_test - test_preds
plt.figure(figsize=(6,4))
plt.hist(residuals, bins=20)
plt.title("Residuals distribution (Test)")
plt.xlabel("Residual")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(6,4))
plt.scatter(y_test, residuals)
plt.axhline(0)
plt.title("Residuals vs Actual G3")
plt.xlabel("Actual G3")
plt.ylabel("Residual")
plt.show()

# Summarize metrics in table form for the report
metrics_df = pd.DataFrame({
    "metric": ["Test RMSE", "Test MAE", "Test R2"],
    "value": [test_rmse, test_mae, test_r2]
})
display(metrics_df)
```